Path Algebras of Quivers as a Sparse Multi-Hop Attention for Mitigating Over-Squashing in GNNs

Carlos Isaac Zainea Maya[1][0000-0002-6459-2397] and Agustín Moreno Cañadas[1][0000-0001-6812-5131]

Universidad Nacional de Colombia, Departamento de Matemáticas, Bogotá, Colombia
{cizaineam,amorenoca}@unal.edu.co

Abstract. Message-passing graph neural networks suffer from *over-squashing*: signal travelling along many long paths into a node is compressed into one fixed-width vector and lost. We introduce **Walk Attention**: the *path algebra* $\Bbbk Q$ of the underlying quiver supplies, through its graded components, the exact multi-hop reachability structure, which we use as the sparse *support* of a learned attention—the algebra says which vertices may interact at range k, the network learns how much. On bottleneck graphs with tunable over-squashing we *prove* the separation—message passing with linear aggregation is at exactly chance accuracy for every depth and width, while a single Walk Attention layer at the matching range solves the retrieval task exactly—and confirm it empirically (1.000 ± 0.000 over five seeds, where one-hop attention degrades toward chance); Walk Attention also stays exact on RingTransfer and, under a shared small budget, ties one-hop attention on Peptides-func, so its advantage is decisive under severe over-squashing and modest when squashing is mild. A negative result sharpens the design: collapsing equivalent walks via the quotient $\Bbbk Q/I$ discards multiplicity the task needs; the algebra's role is to *define the support*, not to compress. Walk Attention is the learned reading of an algebraic substrate whose dynamical reading is the automata-of-impact framework.

Keywords: Graph neural networks · Over-squashing · Path algebra · Quiver · Representation theory · Attention.

1 Introduction

Graph neural networks built on the message-passing paradigm [10] update each node by aggregating messages from its immediate neighbours and iterating this local rule across layers, so that after k layers information from nodes k hops away reaches a target node. While simple and effective, the scheme has a well-documented failure mode: *over-squashing* [3]. When the number of paths carrying information into a node grows—typically exponentially with distance—the

messages along all of those paths must be compressed into a single fixed-width vector; the receiving node cannot recover them individually, and long-range signal is lost. Over-squashing is the principal obstacle to learning tasks whose labels depend on distant or globally-distributed information. Existing remedies act on the graph or on the aggregation: *rewiring* widens bottlenecks [17], curvature and sensitivity analyses explain *where* squashing occurs [6], and a complementary line replaces local aggregation by global Transformer attention [18], at the cost of an all-pairs interaction that ignores graph structure.

In this paper we take a different route, grounded in the representation theory of quivers. A *quiver* is a directed graph, possibly with multiple arrows [15,4], and its *path algebra* $\Bbbk Q$ has the directed walks of the quiver as a basis, with concatenation as multiplication. The graded piece $e_j \cdot \Bbbk Q_k \cdot e_i$ of this algebra has a basis consisting of the directed walks of length k from vertex i to vertex j; its dimension equals $(A^k)_{ij}$, the corresponding entry of the k-th power of the adjacency matrix. The path algebra therefore provides an *exact, algebraic description of multi-hop reachability and multiplicity.*

Our central observation is that this structure is precisely what is needed to define a principled, sparse, multi-hop attention. We let each node attend directly to every vertex reachable from it by a length-k walk: the algebra supplies the *support* of the attention (which pairs may interact at range k), and the network learns the *weights* (how much). Because the walk-reachability support is sparse—typically a small fraction of all node pairs—this is far cheaper than full self-attention, while still acting at a distance and thereby bypassing the iterative bottleneck that causes over-squashing. We call the resulting architecture **Walk Attention**. Since a graph network on Q is a representation of the quiver ($\mathrm{Rep}(Q) \cong \Bbbk Q\text{-Mod}$), Walk Attention is the *learned* reading of an algebraic substrate whose *dynamical* reading is the automata-of-impact-over-quivers framework [23] (Section 3.6).

Contributions.

1. A self-contained account (Section 3) of quivers, path algebras, the impact-degree neighbourhood system, and the *walk operators* encoding multi-hop reachability, with worked examples throughout.
2. **Walk Attention** (Section 4): a learned multi-hop attention whose support is the path-algebra walk-reachability structure, related to message passing and to Transformer attention.
3. A *proved* separation (Section 4.2): with the standard one-layer-per-hop depth, any GNN whose first aggregation is linear in the source features (GCN, GIN) has exactly chance accuracy on the bottleneck family, for every depth, width and parameters, while a single Walk Attention layer at the matching range solves the task exactly—confirmed empirically (Section 5), with a regime-dependent advantage on RingTransfer and Peptides-func.
4. A *negative* result that sharpens the design: collapsing equivalent walks via the quotient $\Bbbk Q/I$ discards needed multiplicity and fails; the quotient's role is to determine the support, not to compress the signal.

2 Related Work

Over-squashing, attention, and multi-hop aggregation. Alon and Yahav [3] identified over-squashing with the NeighborsMatch probe; Topping et al. [17] connected it to negative curvature and proposed SDRF rewiring; Di Giovanni et al. [6] gave a sensitivity analysis relating squashing to width, depth and topology. GAT [19] learns one-hop attention weights; graph Transformers apply all-pairs attention [18] at $O(n^2)$, reinjecting topology through positional encodings. Aggregating beyond one hop is itself well explored: MixHop [2] and SIGN [8] mix fixed adjacency powers, graph diffusion convolution [9] rewires by a diffusion kernel, MAGNA [20] diffuses attention scores, shortest-path message passing aggregates over distance shells [1], NAGphormer [5] tokenises hop-wise aggregates, Graphormer [21] biases dense attention by shortest-path distance, and DRew [11] rewires dynamically with delay. Walk Attention differs in the object defining the support: the *graded path algebra*—walks with their multiplicity, not distance shells or diffusion weights—which makes the support exact (Proposition 1), exposes the quotient $\Bbbk Q/I$ as a design degree of freedom, and embeds the architecture in quiver representation theory.

Path-algebraic structure. The path algebra is a standard object in representation theory [15,4]; Brauer-configuration and related path-algebraic constructions have recently been applied to combinatorial problems by Moreno Cañadas and collaborators [14,13]. Closest to the present work is the impact-degree neighbourhood system and the automata-of-impact framework [23,12], which we take as the dynamical sibling of Walk Attention over the same algebraic substrate (Section 3.6). Attention over multi-hop supports is thus not new in itself; what is new, to our knowledge, is taking the support from the graded path algebra, with its multiplicities, its quotient, and its dynamical reading.

3 The Path Algebra of a Quiver

This section is self-contained: we build the quivers, paths, the path algebra with its grading, the impact-degree neighbourhood system, and the walk operators that the model of Section 4 relies on, assuming no prior exposure. Standard references are [15,4].

3.1 Quivers and paths

Definition 1. *A* quiver *is a quadruple $Q = (Q_0, Q_1, s, t)$ where Q_0 is a finite set of* vertices*, Q_1 a finite set of* arrows*, and $s, t\colon Q_1 \to Q_0$ assign to each arrow α its source $s(\alpha)$ and target $t(\alpha)$; we write $\alpha\colon i \to j$. Multiple arrows between the same ordered pair are allowed. We identify Q_0 with $\{1, \dots, n\}$.*

A quiver is thus a directed multigraph, and for pattern recognition it is exactly the data of a directed relational structure: vertices are entities, arrows are directed relations, and parallel arrows record multiplicity.

Example 1. Let $Q_0 = \{1, 2, 3\}$ with arrows $\alpha, \alpha' \colon 1 \to 2$ (two parallel arrows), $\beta \colon 2 \to 3$, and a loop $\gamma \colon 3 \to 3$. Its adjacency matrix, counting arrows, is

$$A = \begin{pmatrix} 0 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}.$$

In a citation network, for instance, the vertices are documents, an arrow is a citation, the parallel pair α, α' records that document 1 cites document 2 twice, and the loop γ is a self-citation.

Definition 2. *A* path of length $k \geq 1$ *in Q is a sequence of arrows $p = \alpha_k \cdots \alpha_2 \alpha_1$ with $t(\alpha_r) = s(\alpha_{r+1})$, read right-to-left as in function composition, with source $s(p) = s(\alpha_1)$ and target $t(p) = t(\alpha_k)$. Each vertex i also carries a* trivial path e_i *of length* 0.

A path is a directed *walk*: vertices and arrows may repeat. In Example 1, $\beta\alpha$ and $\beta\alpha'$ are two distinct paths of length 2 from 1 to 3, and the loop generates arbitrarily long walks $\gamma^m \beta\alpha$. The number of distinct walks of a given length between two vertices is precisely the quantity that drives over-squashing, and the path algebra makes it algebraically explicit.

3.2 The path algebra and its grading

Let $\Bbbk$ be a field (in practice $\Bbbk = \mathbb{R}$).

Definition 3. *The* path algebra $\Bbbk Q$ *is the $\Bbbk$-vector space with basis the set of all paths of Q, endowed with the bilinear multiplication given on basis paths by concatenation:*

$$q \cdot p \;=\; \begin{cases} q\,p & \text{if } s(q) = t(p), \\ 0 & \text{otherwise.} \end{cases}$$

The trivial paths satisfy $e_{t(p)}\, p = p = p\, e_{s(p)}$ and $e_i e_j = \delta_{ij} e_i$, so $\{e_i\}$ is a complete set of orthogonal idempotents and $1 = \sum_i e_i$ is the unit.

The multiplication respects length: concatenating paths of lengths k and ℓ yields a path of length $k + \ell$ (or zero). Hence $\Bbbk Q$ is a *graded* algebra,

$$\Bbbk Q \;=\; \bigoplus_{k \geq 0} \Bbbk Q_k, \qquad \Bbbk Q_k \cdot \Bbbk Q_\ell \subseteq \Bbbk Q_{k+\ell},$$

where $\Bbbk Q_k$ is spanned by the paths of length exactly k; in particular $\Bbbk Q_0 = \bigoplus_i \Bbbk e_i$ and $\Bbbk Q_1$ is spanned by the arrows. The idempotents let us isolate the paths between a fixed pair of vertices: the subspace $e_j \cdot \Bbbk Q_k \cdot e_i$ is spanned exactly by the length-k paths from i to j, and its dimension is therefore the number of such walks. The following proposition states the correspondence with linear algebra precisely; it underlies everything the model computes.

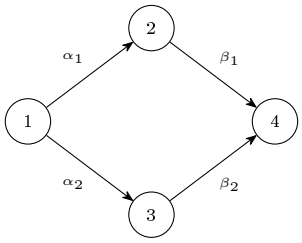

Fig. 1. The diamond quiver of Example 2: two parallel length-2 walks $1 \rightsquigarrow 4$; $\dim(e_4 \Bbbk Q_2 e_1) = 2$.

Proposition 1. *Let $A \in \mathbb{N}^{n\times n}$ be the adjacency matrix of Q, $A_{ij} = \#\{\alpha \in Q_1 : \alpha\colon i \to j\}$. Then for all $k \geq 0$ and all $i, j \in Q_0$,*

$$\dim_{\Bbbk}\big(e_j \cdot \Bbbk Q_k \cdot e_i\big) \;=\; (A^{\,k})_{ij}.$$

Proof. By induction on k; for $k \in \{0, 1\}$ this is the definition. Every length-k path from i to j factors uniquely as $\alpha \cdot p$, where p has length $k-1$ from i to some intermediate vertex r and $\alpha\colon r \to j$ is an arrow; the number of such factorisations is $\sum_r (A^{\,k-1})_{ir} A_{rj} = (A^{\,k})_{ij}$. □

We call $n_k(i, j) := (A^{\,k})_{ij} = \dim_{\Bbbk}(e_j \Bbbk Q_k e_i)$ the *walk multiplicity* from i to j at range k. It is exactly the number of length-k messages that a message-passing GNN forces from node i into node j after k layers—and hence the source of over-squashing.

Example 2. Let Q have vertices $\{1, 2, 3, 4\}$ and arrows $\alpha_1\colon 1 \to 2$, $\alpha_2\colon 1 \to 3$, $\beta_1\colon 2 \to 4$, $\beta_2\colon 3 \to 4$ (Figure 1). Then

$$A = \begin{pmatrix} 0&1&1&0\\ 0&0&0&1\\ 0&0&0&1\\ 0&0&0&0 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 0&0&0&2\\ 0&0&0&0\\ 0&0&0&0\\ 0&0&0&0 \end{pmatrix},$$

and $\dim(e_4 \Bbbk Q_2 e_1) = (A^2)_{14} = 2$, in agreement with Proposition 1: the two *parallel* length-2 walks $\beta_1\alpha_1$ and $\beta_2\alpha_2$, with the same source, target and length. Whether they should be treated as one channel or two is a modelling decision we return to in Section 5.

3.3 Impact degree and the fundamental neighbourhood system

The grading by length induces a canonical stratification of the vertices around each node, which organises the multi-hop structure the model will exploit. The *impact degree* $g(i, j)$ is the length of the shortest directed path from i to j (with $g(i, i) = 0$ and $g(i, j) = \infty$ if no directed path exists); equivalently, $g(i, j) = \min\{k : (A^{\,k})_{ij} > 0\}$, the first range at which the walk multiplicity becomes positive.

Definition 4. *For a node $c \in Q_0$, its* in-neighbourhood *of radius k is $A_k(c) = \{i \in Q_0 : g(i,c) \leq k\}$, the set of nodes that can influence c along a directed path of length at most k. The nested family $\{A_k(c)\}_{k\geq 0}$, with $A_0(c) = \{c\} \subseteq A_1(c) \subseteq \cdots$, is the* fundamental neighbourhood system *(FNS) of c; the layer $\mathcal{N}_k(c) = A_k(c) \setminus A_{k-1}(c)$ collects the nodes at impact degree exactly k.*

The FNS is the combinatorial object that the walk operators below act on, and its precise relation to walk-reachability is the following.

Proposition 2. *For $k \geq 1$ let $\mathcal{S}_k = \{(c,i) : (A^k)_{ic} > 0\}$ be the pairs joined by a length-k walk $i \rightsquigarrow c$. Then*

$$\{(c,i) : i \in \mathcal{N}_k(c)\} \subseteq \mathcal{S}_k,$$

and the inclusion is strict in general: length-k walks may also reach nodes at impact degree $< k$. If Q is graded *—there is $\rho\colon Q_0 \to \mathbb{Z}$ with $\rho(t(\alpha)) = \rho(s(\alpha))+1$ for every arrow α—equality holds for every k.*

Proof. If $g(i,c) = k$, a shortest path is a length-k walk, so $(A^k)_{ic} > 0$. Strictness: in $1 \to 2 \to 3$ with an extra arrow $1 \to 3$, $(A^2)_{13} > 0$ yet $g(1,3) = 1$. If Q is graded, every walk $i \rightsquigarrow c$ has length $\rho(c) - \rho(i)$, so a length-k walk forces $g(i,c) = k$. □

The benchmark graphs of Section 5 are graded (the layer index is such a ρ), so there the supports coincide with the FNS layers; on graphs with cycles, such as a ring, $\mathcal{S}_k$ strictly contains the layer relation. Stacking ranges $k = 1, 2, \ldots$ thus walks an embedding outward through the layers of the FNS, and the multiplicity $n_k(i,c)$ records *how many* length-k paths populate each layer—the textured, algebraic refinement of the scalar impact degree.

3.4 Walk operators

Proposition 1 lets us turn the graded path algebra into a family of linear operators on node features, one per range; these are the bridge to the neural model.

Definition 5. *For each range $k \geq 1$ define $W^{(k)} \in \mathbb{R}^{n\times n}$ by*

$$W^{(k)}_{ji} = \frac{(A^k)_{ij}}{Z_k}, \qquad Z_k = \max_j \sum_i (A^k)_{ij},$$

a single normalising constant per range. The walk-reachability support *at range k is $\mathcal{S}_k = \{(j,i) : (A^k)_{ij} > 0\}$.*

Acting on a feature matrix $H \in \mathbb{R}^{n\times d}$, the operator $W^{(k)}H$ sends to each node j a weighted sum of the features of every node i that reaches j by a length-k walk, the weight proportional to the multiplicity $n_k(i,j)$. Two facts are worth stressing. *(i) Global, not row, normalisation*: row normalisation would map the

multiplicities to a uniform distribution over the reachable sources, erasing exactly the information that distinguishes $W^{(k)}$ from a plain reachability mask; the global constant keeps the relative multiplicities intact while bounding magnitudes. *(ii) Sparsity*: $\mathcal{S}_k$ is the nonzero pattern of A^k, so on the structured graphs of interest $W^{(k)}$ is stored and applied in $O(|\mathcal{S}_k|)$ rather than $O(n^2)$. The fixed-weight $W^{(k)}$ already mitigates over-squashing, because it lets node j read from range-k sources *directly*, in one step, instead of forcing the signal through k rounds of local message passing. Its limitation—which motivates Walk Attention—is that the weights are fixed by multiplicity and cannot *select* which reachable source matters.

3.5 The quotient algebra

Representation theory routinely passes from $\Bbbk Q$ to a quotient $\Bbbk Q/I$ by an *admissible ideal* I, identifying paths that are declared equal. In the diamond of Example 2, the relation $\beta_1\alpha_1 \equiv \beta_2\alpha_2$ generates an ideal for which $\dim\big(e_4(\Bbbk Q/I)_2 e_1\big) = 1$: the two parallel walks collapse to a single class, and in general $\dim\big(e_j(\Bbbk Q/I)_k e_i\big) \leq n_k(i,j)$. It is tempting to use this quotient directly as an over-squashing remedy: replace the walk multiplicities by the smaller quotient dimensions, "deduplicating" redundant paths before aggregation. We tested this and found it *counterproductive* (Section 5): collapsing parallel walks destroys multiplicity information that downstream tasks may require. The constructive role of the quotient is not to compress the signal but to *reshape the support*—for instance routing distinct path classes to distinct attention heads, a direction we leave to future work. In the model that follows the algebra enters only through the support $\mathcal{S}_k$ and the unquotiented operators $W^{(k)}$.

3.6 A common algebraic ground

The objects above—the graded algebra $\Bbbk Q$, the impact degree with its FNS, and the quotient $\Bbbk Q/I$—form one algebraic substrate that specialises in two directions, according to how a node-update map φ over the layers $\mathcal{N}_k(c)$ is read. In the *dynamical* reading, φ is a fixed evolution rule weighted by impact degree; iterating it in time yields the *automata of impact over quivers* (AIQ), with its SI/SIS/SIR rules and category structure, developed separately [23,12]. In the *representational* reading, taken here, φ is *learned*: a vector space at each vertex and a linear map at each arrow is a representation of Q (a $\Bbbk Q$-module, via $\mathrm{Rep}(Q) \cong \Bbbk Q\text{-Mod}$), and a graph neural network on Q is such a representation with learned arrow maps. Walk Attention is its graded instance; the two readings share the trunk and differ only in whether φ is prescribed or learned.

4 Walk Attention

We now define the architecture, and we begin with the observation that motivates it: *the walk operator already is an attention.* An attention layer, in its general

form, fixes a *support* relation S of (target, source) pairs and computes, for each target j, a normalised weighted combination of transformed source features,

$$\mathrm{out}_j \;=\; \sum_{i:\,(j,i)\in S} \alpha_{ji}\, V x_i, \qquad \alpha_{ji} \geq 0, \quad \textstyle\sum_i \alpha_{ji} \text{ bounded}, \tag{1}$$

and architectures differ only in two choices: *which pairs* S contains, and *where the coefficients* α come from. Transformer self-attention [18] takes $S =$ all pairs and learns α from query–key products; GAT [19] takes $S =$ the edges and learns α. Now compare with the walk operator of Definition 5: acting on features it computes $(W^{(k)}H)_j = \sum_{i:(j,i)\in\mathcal{S}_k} \frac{n_k(i,j)}{Z_k} H_i$— *exactly the form* (1), with support the walk-reachability $\mathcal{S}_k$ and coefficients fixed at the normalised walk multiplicities. Aggregating over length-k walks is therefore not "like" attention; it *is* an attention whose support the path algebra determines exactly (Proposition 1) and whose weights happen to be frozen. This dictates the design: keep the algebraic support and replace the frozen multiplicity weights by learned query–key weights—the result is Walk Attention, the learned node update φ of the common substrate of Section 3.6.

4.1 The Walk Attention layer

Fix a range k with support $\mathcal{S}_k$. A Walk Attention layer with H heads of dimension C computes queries, keys and values by linear maps, $Q^h = xW_Q^h$, $K^h = xW_K^h$, $V^h = xW_V^h$, and attends *only over the reachable pairs* $(j,i) \in \mathcal{S}_k$:

$$\ell_{ji}^h = \tfrac{1}{\sqrt{C}}\langle Q_j^h, K_i^h\rangle, \qquad \alpha_{ji}^h = \mathrm{softmax}_{i:(j,i)\in\mathcal{S}_k}\, \ell_{ji}^h, \qquad z_j^h = \textstyle\sum_{i:(j,i)\in\mathcal{S}_k} \alpha_{ji}^h V_i^h. \tag{2}$$

The softmax is taken, for each target j, over exactly the sources that reach j by a length-k walk—a *segmented* softmax over the support, never over all n nodes. The head outputs are concatenated and a residual "root" term is added, $\mathrm{WA}_k(x)_j = \|_h z_j^h + x_j W_{\mathrm{root}}$. Because (2) only ever touches pairs in $\mathcal{S}_k$, the layer costs $O(|\mathcal{S}_k|HC)$ time and memory, not $O(n^2)$; the supports are precomputed once per graph as the nonzero patterns of A^k. Figure 2 shows the data flow and Algorithm 1 summarises one layer.

Example 3. In the diamond of Example 2 at range $k = 2$ the support is the single pair $\mathcal{S}_2 = \{(4,1)\}$: node 4 attends directly to node 1, the softmax over a singleton gives $\alpha_{41} = 1$, and $\mathrm{WA}_2(x)_4 = Vx_1 + x_4 W_{\mathrm{root}}$—the two parallel walks become one *direct* interaction. Adding a second source $1'$ with arrows to 2 and 3 gives $\mathcal{S}_2 = \{(4,1),(4,1')\}$: the softmax now weighs the two sources by the match of their keys against the query Qx_4, so node 4 reads both range-2 sources directly *and chooses between them*—whereas a one-hop network must push both signals through the shared vertices $2,3$, the very compression that over-squashing names.

A depth-L Walk Attention network stacks one layer per range $k = 1,\dots,L$, so successive layers read from progressively longer walks; each layer is followed by

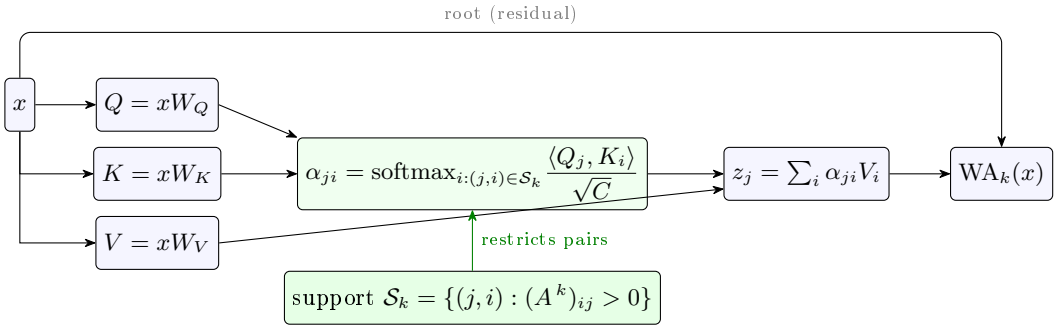

Fig. 2. One Walk Attention layer at range k. Queries, keys and values are linear maps of the node features; attention is computed *only* over the walk-reachable pairs $\mathcal{S}_k$ supplied by the path algebra (the nonzero pattern of A^k), then aggregated and added to a residual root term. The green support box is what distinguishes Walk Attention from dense ($\mathcal{S}_k$ = all pairs) and one-hop ($\mathcal{S}_k$ = edges) attention.

Algorithm 1 Walk Attention layer at range k (sparse).

Require: features $x \in \mathbb{R}^{n\times d_{\text{in}}}$; support $\mathcal{S}_k = \{(j,i) : (A^k)_{ij} > 0\}$

$Q, K, V \leftarrow xW_Q,\ xW_K,\ xW_V$ ▷ per head; shapes $n \times HC$

for each $(j,i) \in \mathcal{S}_k$ and head h **do**

  $\ell^h_{ji} \leftarrow \langle Q^h_j, K^h_i\rangle / \sqrt{C}$

end for

$\alpha^h_{ji} \leftarrow$ segmented-softmax over $\{i : (j,i) \in \mathcal{S}_k\}$

$z^h_j \leftarrow \sum_i \alpha^h_{ji} V^h_i$ ▷ scatter-add into targets

return $\|_h z^h \ + \ xW_{\text{root}}$

layer normalisation, an ELU nonlinearity and dropout, and a final MLP maps the embedding to task outputs. Layer normalisation is important in practice: without it the learned attention is high-variance across initialisations; with it, together with a short learning-rate warmup and gradient clipping, training converges reliably (Section 5).

Table 1 situates Walk Attention among related operators along the two axes of (1). A one-hop GAT layer learns weights but must propagate long-range signal through L stacked layers—precisely the over-squashing regime; a full graph Transformer learns weights over all pairs, paying $O(n^2)$ and discarding structure unless positional encodings reinject it; the walk operator has the right multi-hop support but cannot select. Walk Attention occupies the remaining cell: multi-hop, structure-aware support *and* learned, selective weights.

4.2 A formal account on the bottleneck family

On the bottleneck-chain family used in Section 5, the separation between one-hop message passing and Walk Attention can be *proved*, not only measured. The construction: K sources carry a one-hot *key*—the key assignment is a uniformly

Table 1. Walk Attention against related aggregation operators, by the two axes of (1): support and weights.

Operator	Support	Weights	Cost
GCN / message passing	1-hop	fixed (degree)	$O(\|Q_1\|)$
Graph attention (GAT)	1-hop	learned	$O(\|Q_1\|)$
Walk operator $W^{(k)}$	walk-reachable	fixed (multiplicity)	$O(\|\mathcal{S}_k\|)$
Graph Transformer	all pairs	learned	$O(n^2)$
Walk Attention (ours)	**walk-reachable**	**learned**	$O(\|\mathcal{S}_k\|)$

random permutation π—and a one-hot *value*; d bottleneck layers $L_1, \ldots, L_d$ of M interchangeable, featureless vertices follow, consecutive layers fully connected; a target carries a one-hot *query* q and must output the value of the source whose key is q (label $\pi^{-1}(q)$, chance $1/K$). Networks have the standard depth $L = d + 1$—one layer per hop, as in [3] and in our experiments—and hidden width w.

Proposition 3. *For any message-passing network whose update* $h'_v = U\big(h_v, \mathrm{AGG}\{h_u : u \to v\}\big)$ *is shared across vertices: after every round, all M vertices of each bottleneck layer carry identical states. Hence the entire influence of the sources on the target passes through a single w-dimensional state per layer—while KM^d walks carry it (Proposition 1).*

Proof. Induction on the round: vertices of a layer start equal (zero features) and share their in-multiset (all sources for L_1, all of L_{r-1} for L_r), so shared U, AGG keep them equal. □

Proposition 4. *If moreover messages are linear in the sender's state with content-independent coefficients—as in GCN and GIN—then with $L = d + 1$ layers the target's output depends on the sources only through $\sum_s x_s$, which is the same for every π: the one-hot keys of a permutation sum to the all-ones vector. Expected accuracy is exactly $1/K$ for every d, w, and parameters.*

Proof. Source information entering L_1 at round t needs d more hops, reaching the target at round $t + d \le d + 1$ only if $t \le 1$; at round 1 the message from source s is a fixed linear map of the raw x_s with one shared coefficient (sources are isomorphic in the graph), so L_1's state is a function of $\sum_s x_s$, whose key and value blocks are π-independent. Given q, the label $\pi^{-1}(q)$ is uniform, so a π-independent prediction scores $1/K$. □

Proposition 5. *At range $k = d+1$ the target's support is exactly the K sources (the graph is graded; Proposition 2). With W_Q projecting the query slot scaled by any $\beta > 0$, W_K the key slot, W_V the value slot and $W_{root} = 0$, the output $z_t = \sum_s \alpha_s e_{v(s)}$ of a single Walk Attention layer has its largest entry at the correct value, on every instance, for all d and M.*

Proof. The logit of source s is $\beta\,\mathbf{1}[\pi(s) = q]$, so the softmax puts its largest weight $e^\beta/(e^\beta + K - 1)$ on the unique match s^*; the values being distinct basis vectors, the largest coordinate of z_t is α_{s^*}, at $v(s^*) = \pi^{-1}(q)$. □

Example 4. Take $K = 2$, $M = 2$, $d = 1$ and depth $L = 2$. The two key assignments, $\pi = \mathrm{id}$ and the swap, produce the *same* source sum $\sum_s x_s$ (key and value blocks both $(1, 1)$), so by Proposition 4 a GCN or GIN computes identical outputs on the two instances—whose labels differ: accuracy 1/2, chance. A Walk Attention layer at range 2 attends over $\{s_1, s_2\}$ with logits $\beta\langle e_q, e_{\pi(s)}\rangle$, which distinguish the instances at the first multiplication and select the matching source (Proposition 5).

Remark 1. The propositions delimit the mechanisms seen in Section 5. GAT escapes Proposition 4—its first-round coefficients depend on source content—but must still thread the key–value association through the single-vector channel of Proposition 3, re-encoded $d + 1$ times: empirically it sits above chance and degrades with depth. The fixed-weight walk operator reads the sources directly but with *equal* weights (all multiplicities are M^d), so it must embed the whole key–value multiset in w dimensions rather than select one source: it plateaus in between. Walk Attention both reads directly and selects (Proposition 5), and solves the task.

5 Experiments

We evaluate on a controlled family in which over-squashing is severe and tunable, then on two external benchmarks. All code, configurations and experiment logs are released [24].

Bottleneck-chain retrieval. We instantiate the family of Section 4.2 with K=5, M=4 and depths $d \in \{2, 3\}$: the walk multiplicity into the target is KM^d (Proposition 1), while the deepest support consists of only the K source–target pairs ($5/196 \approx 2.6\%$ of node pairs at d=2, $5/324 \approx 1.5\%$ at d=3). All models use hidden width 16, four heads where applicable, d+1 layers/ranges, and AdamW with a ten-epoch warmup, cosine decay and gradient clipping; Walk Attention adds layer normalisation. We report validation target accuracy (the synthetic task has no separate test split), mean ± std over five seeds. Table 2 lands exactly where Propositions 3–5 put it (Remark 1): GCN and GIN are pinned at chance and sit there empirically; GAT degrades sharply with depth; the walk operator crosses the bottleneck in one step but, weighting all sources by multiplicity alone, cannot select the matching one and plateaus; **Walk Attention solves the task exactly** at both depths over all seeds. Without layer normalisation and warmup its accuracy ranged 0.44–1.00 across seeds; the three standard stabilisers remove this variance entirely—an optimisation artefact, not a property of the operator.

Table 2. Target accuracy (mean ± std over 5 seeds) on bottleneck-chain retrieval, K=5, M=4; chance $1/K = 0.20$; the answer lies d+1 hops away. GCN/GIN sit at chance as Proposition 4 mandates (the residual +0.01–0.02 is best-epoch selection on a finite validation split).

Model	**depth** $d = 2$	**depth** $d = 3$
GCN (one-hop, linear aggregation)	0.21 ± 0.01	0.22 ± 0.01
GIN (one-hop, linear aggregation)	0.21 ± 0.01	0.21 ± 0.01
GAT (one-hop attention)	0.45 ± 0.22	0.29 ± 0.17
Walk operator (fixed weights)	0.62 ± 0.12	0.50 ± 0.12
Walk Attention (ours)	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$

Table 3. RingTransfer accuracy vs. ring size n (distance $n/2$), mean ± std over 5 seeds; chance 0.20.

Model	n=6	n=10	n=14	n=18
GCN	1.00 ± 0.00	1.00 ± 0.00	0.81 ± 0.13	0.53 ± 0.33
GAT	1.00 ± 0.00	1.00 ± 0.00	0.69 ± 0.29	0.29 ± 0.19
Walk Attention (ours)	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$

RingTransfer. The bottleneck family is our own construction; to confirm the effect on an independent benchmark we use RingTransfer [6]: a cycle of n nodes in which a source must transmit its one-hot label (five classes, chance 0.2) to the diametrically opposite target, $n/2$ hops away along the *two arcs* of the ring. Every node has two neighbours, so the number of length-$(n/2)$ walks entering the target—what a one-hop network must compress after $n/2$ layers—grows as $2^{n/2}$, while only the two arc walks carry the signal. All models use hidden width 32, the same optimiser recipe, and $n/2$ layers/ranges. Table 3 reports the results: small rings are easy for every model; from n=14 (distance 7) the one-hop baselines degrade and become highly seed-dependent—at n=18 GAT falls to 0.29 ± 0.19 and GCN to 0.53 ± 0.33—while **Walk Attention stays at** 1.000 ± 0.000 **throughout**. The gap widens with ring size, i.e. with the severity of over-squashing, as predicted.

Collapsing walks hurts. The quotient $\Bbbk Q/I$ of Section 3 suggests an alternative remedy: replace the multiplicities $n_k(i, j)$ by the smaller quotient dimensions, de-duplicating equivalent walks before aggregation. We compare the quotient-built fixed-weight operator to the raw one in an ablation that changes *only* the operator and holds everything else—data, architecture, and the optimiser recipe of Table 2—fixed. Table 4 shows the quotient falling clearly below the raw operator at both depths: collapsing parallel walks discards the source multiplicity that the retrieval task needs. In this regime redundant paths are *signal*, not noise; the algebra's contribution is the support, not compression. Whether a regime exists where the quotient's compression helps—genuinely redundant parallel paths, or path classes routed to distinct heads—remains open.

Table 4. De-duplicating walks via $\Bbbk Q/I$ *hurts*: mean $\pm$ std over 5 seeds, same protocol as Table 2 (the walk-operator row coincides).

Operator	**depth** $d = 2$	**depth** $d = 3$
Quotient $\Bbbk Q/I$ (collapsed walks)	0.39 ± 0.04	0.30 ± 0.09
Walk operator (raw walks)	$\mathbf{0.62 \pm 0.12}$	$\mathbf{0.50 \pm 0.12}$

Table 5. LRGB Peptides-func, test AP (mean $\pm$ std over 4 seeds), shared small budget isolating the operator; published results under the standard 500k protocol are higher for every architecture [7,16].

Model	**test AP**
GCN	0.492 ± 0.011
GAT	0.526 ± 0.004
Walk Attention (ours)	0.526 ± 0.007

LRGB Peptides-func. The synthetic benchmarks are deliberately worst cases; to assess real data we use Peptides-func [7] (10,873/2,331/2,331 molecular graphs, median 138 nodes; multi-label classification, test Average Precision), with the same atom embedding, four layers, hidden width 80, mean pooling and AdamW for every model; Walk Attention uses ranges $k = 1, \ldots, 4$, whose support stays sparse (1–8% of n^2 at range 3). Table 5: Walk Attention exceeds GCN and is statistically indistinguishable from GAT (0.526 ± 0.007 vs. 0.526 ± 0.004 over four seeds). Two caveats frame the comparison. First, all models share a deliberately small budget, so absolute numbers sit below published results (GCN 0.593 under the tuned 500k-parameter protocol [7], higher still in re-evaluations [16]): the comparison is *internal*, isolating the aggregation operator. Second, the tie is not for lack of parallel paths (peptides contain hundreds of vertex pairs joined by several short walks) but because over-squashing is *mild* on these small graphs. The advantage of the multi-hop support is therefore *regime-dependent*: decisive when squashing is severe (Tables 2–3), modest when it is mild, at about $1.6\times$ GAT's wall-clock—far below dense all-pairs attention.

6 Conclusion and Future Work

We introduced Walk Attention, an architecture that mitigates over-squashing by attending over a multi-hop support supplied by the path algebra of the underlying quiver. Proposition 1 gives the support and its multiplicities exactly; Propositions 3–5 prove the separation on the bottleneck family—message passing with linear first aggregation is at chance for every depth and width, while one Walk Attention layer at the matching range is exact—and the experiments confirm it, with a regime-dependent advantage on RingTransfer and Peptides-func. A complementary negative result showed that compressing equivalent walks via the quotient is counterproductive when multiplicity carries signal: the role of

the algebra is to determine the attention support, not to discard information. The construction is deliberately elementary and self-contained, an entry point connecting quiver representation theory to graph representation learning; it is the learned reading of a common substrate whose dynamical reading is the AIQ framework [23]. Several directions follow.

- **From learned weights to dynamics.** Inject the attention weights learned on a task into an AIQ as a data-driven evolution rule, reading off the dynamical effect of each weight, and conversely let AIQ stability and reachability analyses interpret and constrain what Walk Attention has learned—to us the most promising bridge.
- **Benchmarks with severe over-squashing.** Real long-range tasks whose graphs contain acute bottlenecks, compared against rewiring [17], multi-hop [11] and graph-Transformer baselines.
- **Quotient-shaped supports.** Use $\Bbbk Q/I$ to *partition* walks into equivalence classes routed to distinct attention heads, rather than to compress; identifying which relations I are useful, and learning them from data, is an appealing problem.
- **Scaling the support.** Truncate, sample, or learn sparser admissible supports on dense graphs, where the range-k support can itself grow large.

Reproducibility. All graph generators, the walk operators, the Walk Attention layer, the baselines and the experiment logs are released [24]; the path-algebra computations build on the open-source `aiq-quivers` library [22].

References

1. Abboud, R., Dimitrov, R., Ceylan, İ.İ.: Shortest path networks for graph property prediction. In: Learning on Graphs Conference (LoG) (2022)
2. Abu-El-Haija, S., Perozzi, B., Kapoor, A., Alipourfard, N., Lerman, K., Harutyunyan, H., Ver Steeg, G., Galstyan, A.: MixHop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In: International Conference on Machine Learning (ICML) (2019)
3. Alon, U., Yahav, E.: On the bottleneck of graph neural networks and its practical implications. In: International Conference on Learning Representations (ICLR) (2021)
4. Assem, I., Simson, D., Skowroński, A.: Elements of the Representation Theory of Associative Algebras: Volume 1. Cambridge University Press (2006)
5. Chen, J., Gao, K., Li, G., He, K.: NAGphormer: A tokenized graph transformer for node classification in large graphs. In: International Conference on Learning Representations (ICLR) (2023)
6. Di Giovanni, F., Giusti, L., Barbero, F., Luise, G., Lio, P., Bronstein, M.M.: On over-squashing in message passing neural networks: The impact of width, depth, and topology. In: Proceedings of the 40th International Conference on Machine Learning (ICML). PMLR, vol. 202, pp. 7865–7885 (2023)
7. Dwivedi, V.P., Rampášek, L., Galkin, M., Parviz, A., Wolf, G., Luu, A.T., Beaini, D.: Long range graph benchmark. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track. vol. 35 (2022)

8. Frasca, F., Rossi, E., Eynard, D., Chamberlain, B., Bronstein, M., Monti, F.: SIGN: Scalable inception graph neural networks. arXiv preprint arXiv:2004.11198 (2020)
9. Gasteiger, J., Weißenberger, S., Günnemann, S.: Diffusion improves graph learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)
10. Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning (ICML). PMLR, vol. 70, pp. 1263–1272 (2017)
11. Gutteridge, B., Dong, X., Bronstein, M.M., Di Giovanni, F.: DRew: Dynamically rewired message passing with delay. In: International Conference on Machine Learning (ICML) (2023)
12. Ibáñez Huertas, J.A., Zainea Maya, C.I.: CAsimulations: Modelado de la dinámica topológica en una enfermedad mediante autómatas celulares. Comunicaciones en Estadística **18**(1) (2025). https://doi.org/10.15332
13. Moreno Cañadas, A., Rodríguez-Nieto, J.G., Salazar Díaz, O.P.: Brauer configuration algebras induced by integer partitions and their applications in the theory of branched coverings. Mathematics **12**(22), 3626 (2024). https://doi.org/10.3390/math12223626
14. Osorio Angarita, M.A., Moreno Cañadas, A.: Brauer configuration algebras for multimedia based cryptography and security applications. Multimedia Tools and Applications **80**(15), 23485–23510 (2021). https://doi.org/10.1007/s11042-020-10239-3
15. Schiffler, R.: Quiver Representations. Springer (2014)
16. Tönshoff, J., Ritzert, M., Rosenbluth, E., Grohe, M.: Where did the gap go? Reassessing the long-range graph benchmark. Transactions on Machine Learning Research (2024), arXiv:2309.00367
17. Topping, J., Di Giovanni, F., Chamberlain, B.P., Dong, X., Bronstein, M.M.: Understanding over-squashing and bottlenecks on graphs via curvature. In: International Conference on Learning Representations (ICLR) (2022)
18. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30 (2017)
19. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. arXiv preprint arXiv:1710.10903 (2017)
20. Wang, G., Ying, R., Huang, J., Leskovec, J.: Multi-hop attention graph neural networks. In: International Joint Conference on Artificial Intelligence (IJCAI) (2021)
21. Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., Liu, T.Y.: Do transformers really perform badly for graph representation? In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
22. Zainea Maya, C.I., Moreno Cañadas, A.: aiq-quivers: a Python library for quivers, path algebras, and impact automata. https://github.com/Izainea/aiq-quivers (2026), version 1.2.1
23. Zainea Maya, C.I., Moreno Cañadas, A.: Automata of impact over quivers (2026), manuscript in preparation
24. Zainea Maya, C.I., Moreno Cañadas, A.: path-algebra-gnn: Walk attention — code and experiments. https://github.com/Izainea/path-algebra-gnn (2026)